

MQTT QoS 0 vs 1 Message Loss under Controlled Disconnect on AWS IoT Core

Uday Kiran Reddy Dodda | Student ID: X25166484

MSc Cloud Computing — National College of Ireland — Short Explanation Guide

Baseline: Shvaika et al. (2025) J. Big Data — TBMQ — DOI 10.1186/s40537-025-01271-x

Read aloud in simple English. Numbers come from STATUS.md / results in the repo — do not invent live AWS metrics in class.

1. What is the project?

This project measures how MQTT Quality-of-Service (QoS) level 0 versus level 1 affects telemetry message loss on Amazon Web Services IoT Core when the publishing device is deliberately disconnected for controlled periods. Internet-of-Things devices often use MQTT because it is light and works over unreliable links. Cloud teams then ask a practical question: if the radio drops for fifteen seconds or five minutes, does QoS 1 actually save messages that QoS 0 would lose — and what latency or backlog cost do you pay on a managed broker?

The formal artefact is `mqtt-qos-iot-core/` only. Do not present the proxy `iot-reliability/` federated-RF tree (or Et-Tousy mapping) as this CA2. Simulated devices publish synthetic telemetry. A device-side ID log is written before each publish attempt. AWS IoT Core rules forward arrivals to a Lambda ingest function, which stores delivered records in DynamoDB. Loss, duplicate-id rate, and end-to-end latency come from matching the device log to DynamoDB. Experimental factors: $\text{QoS} \in \{0,1\} \times \text{disconnect} \in \{0, 15 \text{ s}, 1 \text{ m}, 5 \text{ m}\} \times \text{rate} \in \{\text{steady}, \text{bursty}\}$.

Lite live evidence (2026-09-21): full 16-cell factorial in eu-west-1, about 6 000 IoT messages upper bound (~2.4% of monthly free tier), then Terraform destroy (43 resources). Formal-scale design (~600 000 messages, 5 replications) remains free-tier blocked as a single-month apply and is documented as beyond the CA2 floor. CA2 research-scope alignment is 100%. One-sentence pitch: “I disconnect publishers on purpose on AWS IoT Core and measure whether MQTT QoS 1 really loses fewer messages than QoS 0 — with per-message ground truth.”

2. Problem statement + real-life example

MQTT QoS 0 is fire-and-forget: the client publishes and does not wait for a broker acknowledgement. If the link is down, that message can vanish. QoS 1 requires at-least-once delivery semantics: the client expects an acknowledgement and may retransmit, so you expect lower loss but higher latency and possible duplicates. On a managed broker such as AWS IoT Core, the tenant does not tune Kafka or Redis inside the broker — so self-hosted throughput papers do not answer the disconnect-loss question for the service teams actually buy.

Real-life example — courier and warehouse desk. Each telemetry message is a parcel. Before the courier leaves the van, they stamp the tracking ID in their own book (device-side ID log). QoS 0 is tossing the parcel toward the public desk and driving away — if the road is closed, the parcel never arrives. QoS 1 is waiting for a signed receipt; when the road reopens, a backlog can still flush to the desk. AWS IoT Core is that public warehouse desk: you rent it, you do not own the sorting machines. Shvaika's TBMQ paper studies a self-hosted desk under steady high load and explicitly leaves varying QoS, intermittent connectivity, and varying client conditions as future work — that stated omission is my gap.

Industry angle: fleets, factories, and smart-home products publish through managed MQTT. Saying “just use QoS 1” without measuring loss under cut links on the managed broker is a guess, not an evidence-based setting. QoS 2, multi-region, and physical radios are deferred in this formal CA2 and should not be claimed in the viva.

3. Baseline paper

Shvaika, A., Shvaika, D., Landiak, D. and Artemchuk, V. (2025), “A distributed architecture for MQTT messaging: the case of TBMQ”, *Journal of Big Data* 12(1), 224 (DOI 10.1186/s40537-025-01271-x). They characterise a self-hosted, Kafka/Redis-backed MQTT broker (TBMQ) under steady high load, focusing on delivery completeness and operational behaviour of a broker the operator controls. A companion paper discusses architectural enhancements and replay paths while clients remain connected.

Their gap for this thesis: they name as future work varying QoS levels, intermittent connectivity, and varying client conditions. No reviewed study jointly combines (a) a managed broker the tenant does not operate, (b) controlled publisher disconnection, (c) more than one MQTT service level, and (d) a per-message device-side log as ground truth. Shvaika is the named baseline because it states the omitted conditions clearly. Do not use Et-Tousy et al. federated-RF as this CA2 baseline — that mapping belonged to the quarantined proxy and answers a different research question.

Honest contrast claim: this thesis does not reproduce TBMQ throughput figures. Different research question, different broker, different dependent variable. We invert the setting to managed IoT Core under disconnect and report loss as the primary metric, with latency and cost as the price of reliability.

4. How we solve it / research question

Research question: To what extent does publishing at MQTT QoS level 1 rather than level 0 reduce telemetry message loss in AWS IoT Core when the publishing device undergoes controlled disconnections of varying duration?

Solution: hold the managed broker fixed; inject controlled publisher disconnect windows in the simulator; match device IDs to DynamoDB deliveries; measure loss rate (primary),

duplicate-id rate, end-to-end latency (mean / p95 / p99), reconnection time and backlog survival, plus estimated USD from published unit prices × counted operations. Local mock runs prove the harness only; results/mock/ must never be cited as AWS. Only results/live/ is IoT Core evidence.

Lite live numbers bound in STATUS.md / BASELINE_PAPER.md (do not invent others): no disconnect → QoS0 and QoS1 loss = 0.00; 15 s disconnect → QoS0 loss = 0.30, QoS1 loss = 0.00; 60 s and 300 s under the lite wall-clock schedule → QoS0 loss = 0.68, QoS1 loss = 0.00 (QoS1 pays latency/backlog, mean roughly 3–14 s under disconnect cells). Limitation retained for honesty: n=1 replication per cell; d60≐d300 QoS0 loss under lite schedule because the cut covers remaining publishes — directional floor evidence, not formal Holm power.

5. Dataset / data (professor cares about this)

Fully synthetic device telemetry generated by the artefact. No real factory sensors. No personal data. The formal CA2 contract explicitly allows synthetic devices, because the research object is broker behaviour under controlled disconnect, not a field deployment of physical radios.

- Device-side ID log written before each publish = ground truth of intent to send.
- Lite live (2026-09-21): 16 cells; 5 devices × 50 messages; ≈6 000 IoT messages upper (~2.4% of 250 000 monthly free-tier messages); region eu-west-1; stack destroyed (43 Terraform resources).
- Smoke archive: 4 cells, 2 devices × 8 messages — early harness proof only.
- Mock dry-run under results/mock/ is harness proof — never cite as IoT Core evidence.
- Formal design scale (5 devices × 1000 msgs × 5 replications ≈ 600 000) remains free-tier blocked as a single-month apply — documented beyond-floor work.
- Primary live pointer: [mqtt-qos-iot-core/results/live/](#) (lite campaign + smoke archive).

6. Implementation + technologies / AWS services

- AWS IoT Core — managed MQTT broker (tenant does not operate broker internals).
- IoT Topic Rules — route telemetry into the ingest path.
- AWS Lambda — ingest function that writes each delivered record.
- Amazon DynamoDB — delivered-message store used for matching against the device log.
- Terraform under terraform/ — provision and destroy; enable_apply gate; required project tags.
- Python simulator (src/simulator/) — synthetic devices, disconnect windows, steady/bursty rates, mock and live publishers.

- Matching + analysis (src/matching/, src/analysis/) — loss/dup/latency; cost surface; Holm analysis on mock.
- eu-west-1 lite campaign followed by mandatory scripts/destroy_stack.sh.

7. End-to-end flow

- Pass READY_FOR_AWS gates (tags, destroy hook, free-tier plan for lite/smoke only).
- Terraform apply IoT Core + rule + Lambda + DynamoDB with lite/smoke variables.
- Start simulated devices; stamp message IDs into the device-side log before publish.
- Publish at QoS 0 or 1 under steady or bursty rate.
- Inject controlled disconnect (0 / 15 s / 1 m / 5 m), then reconnect.
- IoT rule → Lambda → DynamoDB for each message the broker delivers.
- Match device log ↔ DynamoDB → loss, duplicates, latency, backlog metrics.
- Write live evidence; Terraform destroy; keep only results under results/live/.

8. Exact speaking script (dataset + implementation + technologies)

“My data is synthetic IoT telemetry from simulated devices — no real sensors and no personal information. Before every publish I write the message ID on the device side, then I match that ID to what actually lands in DynamoDB after AWS IoT Core rules invoke my Lambda ingest function. On the twenty-first of September I ran a lite sixteen-cell live campaign in eu-west-1 — about six thousand messages, under two percent of the monthly free tier — and then I destroyed the forty-three Terraform resources. I compare MQTT QoS 0 versus QoS 1 under controlled disconnect windows of zero, fifteen seconds, one minute, and five minutes, at steady and bursty rates. Technologies are Python, Terraform, AWS IoT Core, Lambda, and DynamoDB. My baseline is Shvaika et al. 2025 on self-hosted TBMQ; the gap is managed broker plus disconnect plus multi-QoS with per-message ground truth. The old federated-RF proxy is not this project.”

9. STATUS snapshot + five likely Q&A

STATUS snapshot (say this if asked “are you finished?”): INITIAL_EVAL_PASS=yes (lite 16-cell); CA2 align 100; READY_FOR_AWS=YES for lite/smoke; formal ~600 k msgs beyond floor / free-tier blocked; stack destroyed; iot-reliability = PROXY only.

Q: Is this the same experiment as Shvaika’s TBMQ paper?

A: No — they self-host under steady load; I measure managed IoT Core under controlled disconnect at QoS 0 and 1.

Q: Do you use real device or customer data?

A: No — synthetic telemetry only; the formal CA2 contract allows synthetic devices.

Q: Is the lite 16-cell run the formal 600 000-message study?

A: No — lite is the free-tier research-floor evidence; formal scale is beyond floor / free-tier blocked.

Q: Did QoS 1 remove loss in the lite campaign?

A: In the lite cells QoS1 loss stayed 0.00 while QoS0 rose under disconnect — reliability is paid for with latency and backlog.

Q: Is `iot-reliability/` your formal artefact?

A: No — that proxy is quarantined. Formal artefact is only `mqtt-qos-iot-core/`.